INFORMATION SECURITY · PROJECT PROPOSAL

USB Analyzer

A cross-platform tool for Windows and Linux that detects, fingerprints and inspects every USB device plugged in, and blocks the dangerous ones.

Team: Muhammad Suleman Tariq ·Muhammad Usman Ansari · Muhammad Jalal Khan · Farhad Khan · Muhammad Shayan

BS Computer Science · NUST · Fall 2026

01 · THE PROBLEM

Operating systems trust whatever is plugged in

Malware on storage

Infected flash drives spread worms between machines. Stuxnet reached air-gapped systems this way.

Keystroke injection (BadUSB)

A device posing as a keyboard types hundreds of commands per second. The OS accepts it with no prompt.

Spoofed identities

Vendor and product IDs are self-reported, so a malicious gadget can claim to be a trusted brand.

No visibility

Most users and labs keep no record of which devices connected, when, or what they carried.

02 · OUR SOLUTION

Five things USB Analyzer does on every plug-in

- **Detect** — catch connect and disconnect events in real time.
- **Fingerprint** — read vendor, product, serial, class and interfaces.
- **Scan** — hash files on storage devices and match them against YARA rules.
- **Flag** — spot keyboard-like devices that type at inhuman speed.
- **Enforce** — allow whitelisted devices, block the rest, log everything.

03 · SCOPE

Sized for five sophomores and four weeks

In scope

- Windows 10/11 (demo) and Ubuntu Linux
- Storage and keyboard (HID) devices
- Whitelist policy with allow / block
- Web dashboard, alerts and PDF report
- Python, open-source libraries only

Out of scope

- macOS
- Kernel drivers or firmware analysis
- Hardware USB protocol sniffers
- Full antivirus engine (we use YARA)
- Enterprise multi-host deployment

04 · FUNCTIONAL REQUIREMENTS

What the system must do

ID	Requirement	Owner module
FR-1	Detect USB connect / disconnect within 1 second, Windows and Linux	Device Monitor
FR-2	Record VID, PID, serial, manufacturer, class and interfaces	Device Monitor
FR-3	Hash every file on a mounted drive (SHA-256)	Storage Scanner
FR-4	Match files against YARA rules; flag autorun and risky extensions	Storage Scanner
FR-5	Flag HID devices whose typing speed exceeds a human threshold	HID Detector
FR-6	Allow whitelisted devices; disable unknown or flagged ones	Policy Engine
FR-7	Store all events in SQLite and expose them through a REST API	Core Engine, API
FR-8	Show live alerts and export a PDF incident report	Dashboard

Non-functional requirements and tech stack

Quality targets

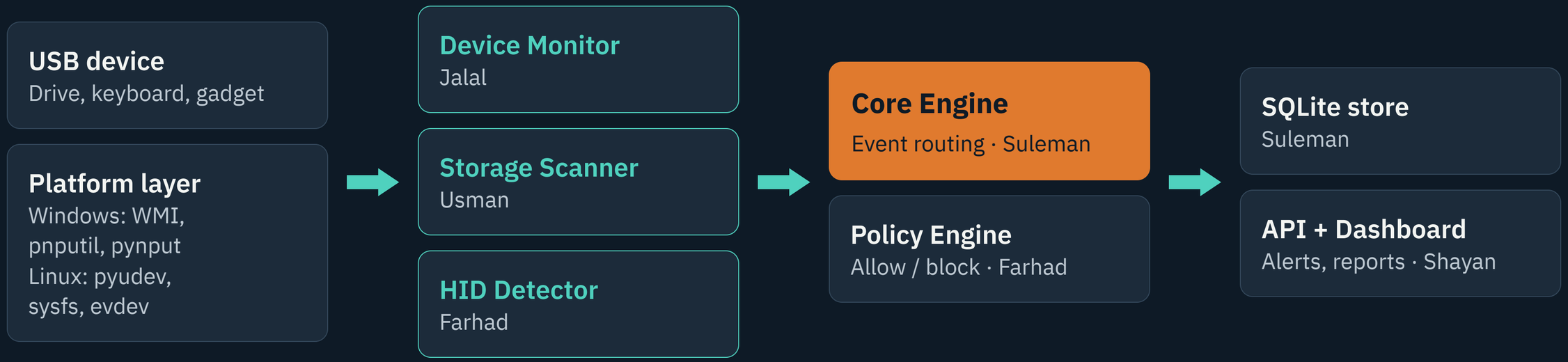
- **Fast:** alert shown within 2 s of plug-in
- **Light:** under 5% CPU while idle
- **Safe:** read-only scanning
- **Auditable:** every decision logged
- **Portable:** one core, Windows and Linux
- **Usable:** no command line needed

Tech stack

Layer	Windows	Linux
USB events	WMI (wmi)	pyudev, sysfs
Keystrokes	pynput	python-evdev
Blocking	pnputil	sysfs authorized
Scanning	hashlib, yara	same
Backend	FastAPI, SQLite	same
Frontend	HTML, JS, Chart.js	same
Teamwork	GitHub, pytest	same

06 · ARCHITECTURE

Only the platform layer changes between Windows and Linux



07 · TEAM AND OWNERSHIP

One module each, plus one shared duty

Member	Module owned	Shared duty
Muhammad Suleman Tariq (Group Leader)	Project foundation and Core Engine	Integration lead
Muhammad Usman Ansari	Storage Scanner (hashing, YARA)	Safe test samples
Muhammad Jalal Khan	Device Monitor and fingerprinting	Lab setup
Farhad Khan	HID Detector and Policy Engine	Attack simulation
Muhammad Shayan	API, Dashboard and Reports	Documentation, final report

Suleman starts the project: his interface and event format, due 15 October, unblock everyone else. Usman's module has the least OS-specific code, so he backs up Farhad on the whitelist.

08 · MODULE BREAKDOWN

What each member builds

Foundation · Suleman

- Repo, interface, event format
- Core Engine and SQLite
- Merge and run all modules

Storage Scanner · Usman

- SHA-256 hashing in chunks
- Known-malware hashes, YARA
- Flag autorun, .exe, .lnk, macros

Device Monitor · Jalal

- WMI and pyudev plug events
- Vendor, product, serial, type
- Spoofed-ID alert

HID and Policy · Farhad

- Keystroke timing (pynput, evdev)
- Whitelist by serial number
- Block: pnputil or sysfs

Dashboard · Shayan

- FastAPI routes
- Live alerts, history, charts
- One-click PDF incident report

Shared contract

Every module emits the same JSON event: time, device ID, source, severity, details. Fixed in week 1, so nobody waits on anyone.

09 · COMPLETION PLAN

Four weeks, demo on 9 November

12 – 18 OCT

Week 1 · Setup

- Repo and interface
- Fix JSON event format
- Each member: library spike
- Proposal approved

19 – 25 OCT

Week 2 · Build

- Modules work alone
- Windows backends
- Unit tests per module
- Dashboard on mock data
- Mid-point check-in

26 OCT – 1 NOV

Week 3 · Integrate

- Modules wired to core
- Linux backends
- Dashboard on real data
- First end-to-end run

2 – 8 NOV

Week 4 · Test and ship

- Attack simulations
- Bug fixes, tuning
- Final report and slides
- Demo rehearsal

Five attack scenarios we will prove live

Scenario	How we simulate it	Expected result
Trusted drive	Whitelisted flash drive, clean files	Allowed and logged
Infected drive	EICAR test file plus a fake autorun.inf	High-severity alert
Keystroke injection	Digispark / Arduino Leonardo, or uinput script	Flagged and blocked
Unknown device	Any device not on the whitelist	Disabled at once
Spoofed identity	Trusted VID/PID, different serial	Mismatch alert

Each module also ships unit tests. Success means all five scenarios pass in one uninterrupted demo run.

What could go wrong, and our plan for it

Risk	Mitigation
Windows blocks after connect, not before	State it openly; Linux blocks by default (authorized_default = 0)
Keyboard hook can't name the keyboard	Link fast typing to the keyboard WMI just reported
No BadUSB hardware in time	Order a Digispark in week 1; uinput fallback on Linux
Typing-speed false alarms	Tune the threshold on all five members' typing
Integration slips	Suleman's interface fixed by 15 Oct; mock events until then
Member unavailable	Every module documented; Usman backs up Farhad

Ethics: defensive tool only. Tests run on our own machines with EICAR and harmless payloads. No real malware.

By 9 November you will see a working USB Analyzer

- Source code on GitHub
- Live five-scenario demo
- Technical report
- User guide

We request approval to begin on 12 October. Questions?